

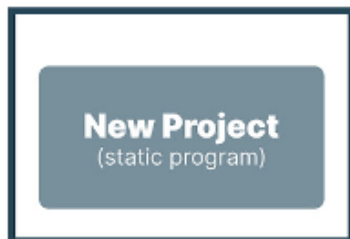
Program

컴퓨터에서 실행할 수 있는 파일(.py, .exe).

Process

- 프로그램 실행에 필요한 모든 자원을 메모리에 올리고, CPU 자원을 할당받은 상태
- 정적인 프로그램이 동적으로 변하여 실행되고 있는 상태
ex) Actor-Critic Program → Actor-Critic Train Process, Actor-Critic Test Process

Program



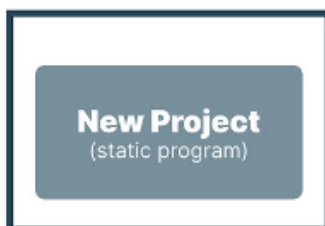
Process



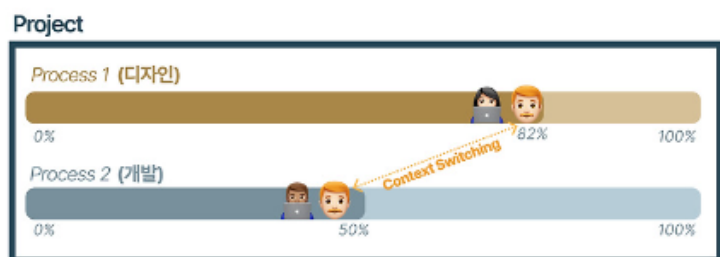
Multi-process

- 프로그램이 여러 개 작동된다는 의미이다.
ex) 카카오톡, Chrome, Vscode, ... 동시에 사용
Actor-Critic Program → train process 4개, test process 1개

Program

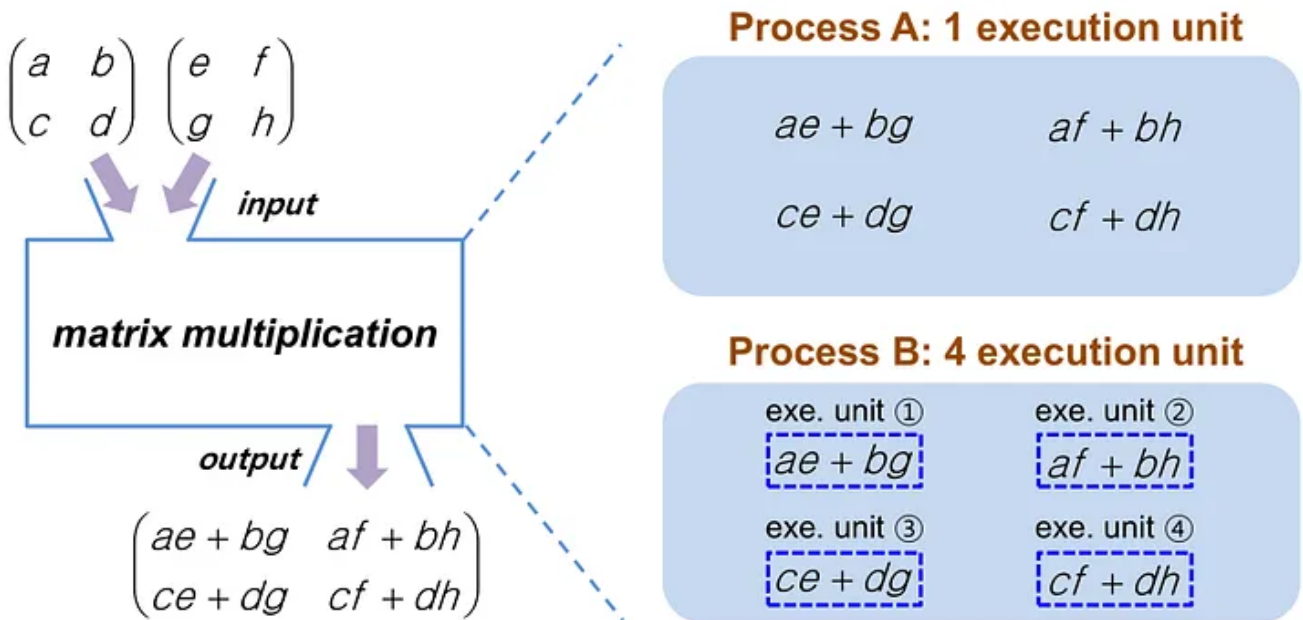


Multi Process



Thread

- 하나의 *Process* 내에서 논리적인 작업을 처리하는 단위
- *하나의 Process*는 반드시 *하나 이상의 Thread*를 갖는다.
- 하나의 *Process*가 두 개의 *Thread*를 가질 때, 이 두 개의 *Thread*는 *Process*의 자원을 공유하면서 *Process* 실행 흐름의 일부가 된다.
ex) 8 Thread → 최대 8개의 *Process*를 동시에 처리할 수 있다.



Process A 는 *한 개*의 thread 상에서 연산을 수행한다.

Process B 는 *네 개*의 thread 상에서 연산을 수행한다.

multi-thread 는 여러 개의 thread 가 역할을 나누어서 process 를 처리할 수 있다.

CPU (Central Processing Unit)

- 기억, 연산, 제어 등 컴퓨터의 논리적 사고를 담당하는 **두뇌 역할**을 담당한다.
- CPU가 1개 운용된다면 *Single CPU*, 여러 개가 운용된다면 *Multi-CPU*라고 한다.
- CPU는 Process를 실행하고 관리한다.

Core

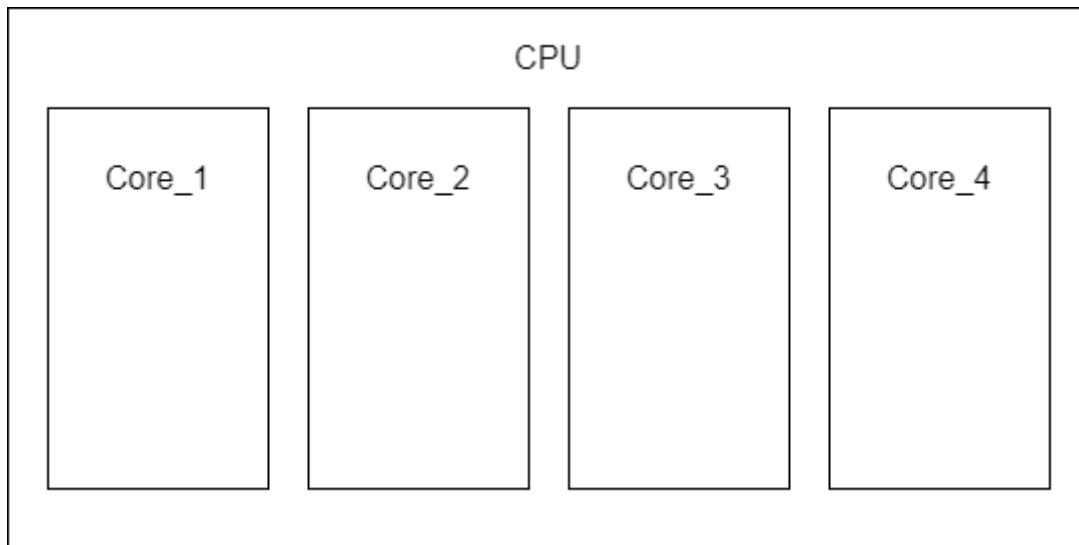
- CPU의 **물리적** 구성 요소로, 컴퓨터의 연산 처리 속도를 결정하는 중요한 요소
- 보통 CPU에는 여러 개의 Core가 존재하고, 각 Core는 **독립적으로** 작업을 수행한다.

ex) CPU : 회사 사장 / Core : 직원

Multi-core

ex) 2코어 (듀얼 코어), 4코어 (쿼드 코어), 8코어 (옥타 코어)

4코어 (쿼드 코어)



Thread

- 하나의 *Process* 내에서 논리적인 작업을 처리하는 단위
- 하나의 Process는 반드시 하나 이상의 Thread를 갖는다.
- 데이터의 실행 흐름을 나타내며, 데이터가 오가는 통로

원래 하나의 Core에 하나의 Thread만 존재했지만, 하이퍼스레딩이라는 기술이 생기면서 하나의 Core에 여러개의 Thread가 존재하게 되었다.

ex) 4 Core 8 Thread

- 4 코어: 물리적 코어 개수, 8 Thread: 논리적 코어 개수
- 최대 8개의 작업(Process)를 동시에 처리할 수 있다

4 Core 4 Thread의 CPU 환경에서 16개의 작업이 있는 상황

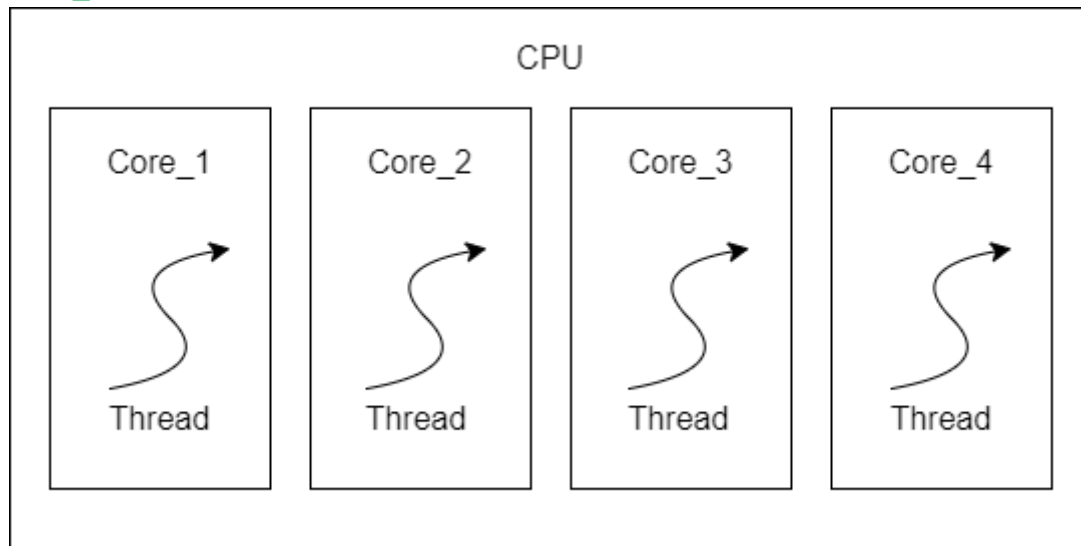
동시성 (Concurrency)

CPU의 Core가 같은 시간에 여러 다른 작업을 *처리하는 것처럼* 보이는 것을 뜻한다.

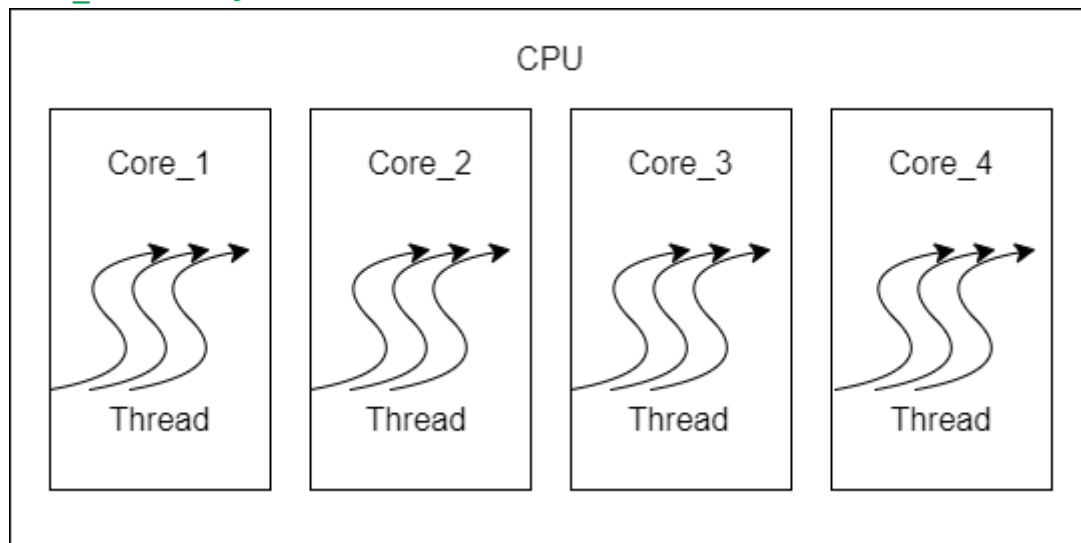
병렬성(parallelism)

같은 시간에 여러 다른 작업을 *실제로 처리*하는 것

Num_Threads = 1



Num_Threads = 3



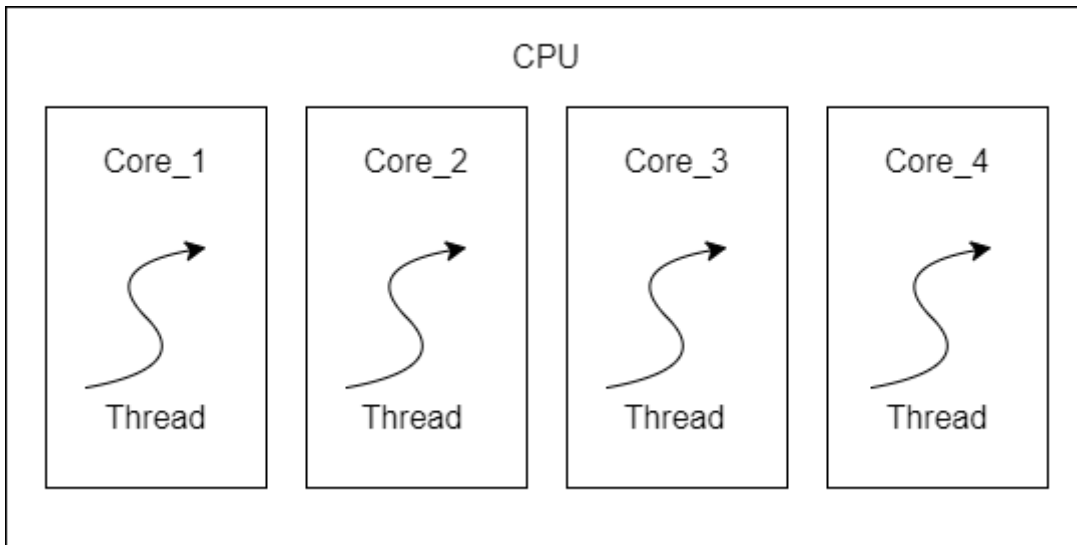
Multi-Thread 란 하나의 CPU 안에 작업의 줄기가 여러 개 있다는 뜻이다.

ex) 1대의 모니터당 1개의 업무 처리를 할 수 있다고 가정하자.

따라서, 여러 대의 모니터를 보유함으로써 다중적 업무 처리가 가능해진다.

- CPU(Processor) : 논리적 사고를 담당하는 회사 사장
- Core : 연산 업무를 처리하는 직원
- Thread : 직원이 업무를 처리하는 모니터

→ **single multi-core CPU & num_thread = 1 :**



A3C

A3C에서 multiprocessing 을 활용하여 여러 개의 workers가 동시에 작업을 수행할 수 있도록 한다.

이번 코드에서는 4개의 train process와 1개의 test process를 생성한다.

1. Train process

- **worker**를 학습시키는 역할을 한다.
- 각 프로세스는 독립된 환경에서 학습을 수행하고, 중앙의 공유 모델을 주기적으로 갱신한다.
- 4개의 Train Process를 생성한다 = Worker의 개수가 4개다

3. Test process

- 주기적으로 **현재 모델의 성능을 평가**한다.
- worker를 학습시키지 않고, 모델의 성능을 평가하기 위해 환경을 실행한다.

Conclusion

4 train processes & 1 test process & a single multi-core CPU & num_thread = 1

Discussion

1. Train Process의 개수 = Worker의 개수 (o)
2. Process의 개수 = Core의 개수 (x)
3. Process의 개수 = Thread의 개수 (x)